College Registration System Database Design

Design Activity & Practical: from requirements to a relational schema

Presented by: **SHEMA SEBATUNZI Samuel** 21 October 2026

Requirements | Conceptual ERD | Relational tables | Data types | Draw.io

1. Requirements: users & processes

Who uses the system, and what do they do?

User	Needs
Student	Register / drop courses, registration slip, view grades
Registrar	Enrol students, maintain curricula, approve registrations
Lecturer	Class lists, enter final grades
Head of Dept.	Assign courses to lecturers, statistics
Sys. Admin	Accounts, backups, security

1. Set up departments, programmes & curricula
2. Allocate a lecturer to each course
3. Enrol student into one programme
4. Student registers for courses → *Pending*
5. Registrar approves → *Approved* (or student *Drops*)
6. Lecturer enters grade → *Completed*
7. Reports: class lists, slips, pending approvals

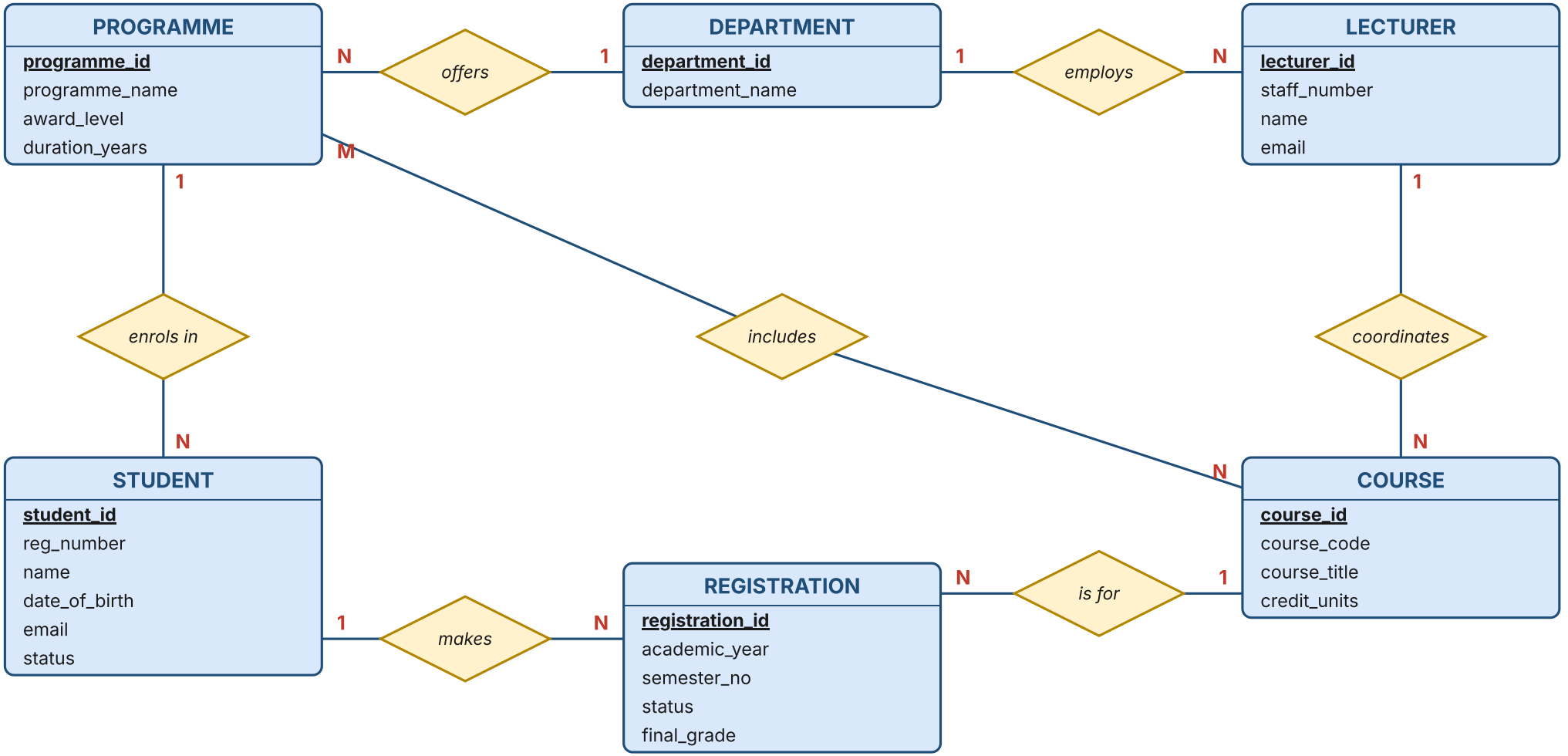
2. Entities & business rules

Example entities: Student, Programme, Course, Registration and Lecturer (plus Department)

#	Business rule	Enforced by
BR1	A department offers many programmes and employs many lecturers	FK `department_id` (NOT NULL)
BR2	A student is enrolled in exactly one programme	FK `student.programme_id`
BR3	A programme has many courses; a course can be in many programmes (M:N)	Associative table `PROGRAMME_COURSE`
BR4	Each course is coordinated by exactly one lecturer	FK `course.lecturer_id`
BR5	A student registers for many courses, but for each course only once per semester	`REGISTRATION` + UNIQUE(student, course, year, semester)
BR6	Only Active students; only courses in their programme; at most 24 credits per semester	Application check + audit query
BR7	Status Pending / Approved / Dropped / Completed; grade only when Completed	CHECK constraints

3. Conceptual model (ERD)

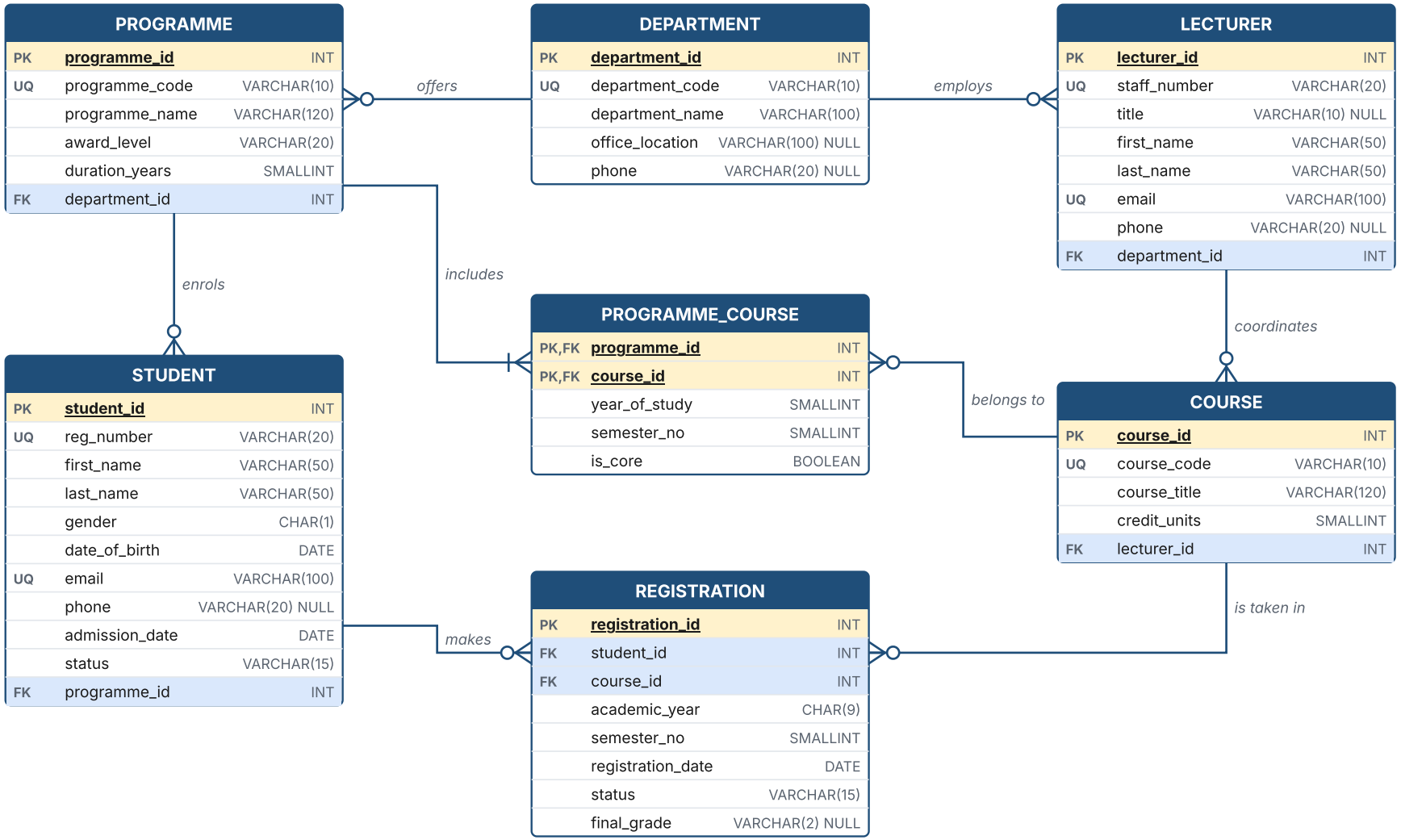
Chen notation, drawn in Draw.io: entities, relationships and cardinalities



Two M:N relationships: Programme–Course ("includes") and Student–Course (through REGISTRATION)

4. Relational schema & data types

7 tables · every table has a primary key · each 1:N is a foreign key · each M:N is an associative table



5. Implementation & checks

The schema was built and tested in SQL (sql/01_schema.sql)

```
CREATE TABLE registration (
  registration_id   INT         NOT NULL,
  student_id        INT         NOT NULL,
  course_id         INT         NOT NULL,
  academic_year     CHAR(9)     NOT NULL,
  semester_no       SMALLINT    NOT NULL,
  registration_date DATE        NOT NULL,
  status            VARCHAR(15) NOT NULL,
  final_grade       VARCHAR(2)  NULL,
  PRIMARY KEY (registration_id),
  UNIQUE (student_id, course_id,
          academic_year, semester_no),
  FOREIGN KEY (student_id)
    REFERENCES student (student_id),
  FOREIGN KEY (course_id)
    REFERENCES course (course_id)
);
```

Tests run against the database:

- Duplicate registration in the same semester: **rejected**
- Registration for a non-existent student: **rejected**
- Invalid gender value: **rejected**
- Grade on a registration that is not Completed: **rejected**
- Deleting a department that still has programmes: **rejected**
- Courses outside the student's programme: **0 rows**

Normalised to 3NF: no repeating groups and no partial or transitive dependencies.